CP468 Assignment 2 Report: Connect Four AI

CP468-D — Artificial Intelligence

Wilfrid Laurier University

Group 7

Students:

Farhan Chowdhury

Manahil Bashir

Morad Mohammad

Noah Samarita

Safdar Md Abdus Shukur

1. Introduction

Connect Four is a simple game on the surface, but it hides a lot of depth once you start thinking about search and strategy. This assignment asked us to build three agents of increasing intelligence and see how they actually perform against each other, rather than just talking about theory. Our focus was on getting the game engine correct, then layering three agents on top of it: a random baseline, a rule-based agent, and a minimax agent with alpha-beta pruning. The point of the assignment wasn't a polished interface, it was showing that the search and heuristic techniques from class actually work when tested head to head.

2. System Design

The core of the project is a single engine file that all three agents share, so none of them have their own private version of the rules. The board is stored as a 6 by 7 grid, with row 0 as the bottom row. Dropping a disc into a column finds the lowest open row in that column, which handles the "gravity" requirement. The engine also exposes a clean set of functions that any agent can call: getting the list of legal moves, applying a move, checking whose turn it is, and checking for a winner. There's also a clone function that makes a full copy of the board state, which matters a lot for the minimax agent since it needs to simulate moves ahead without touching the real game.

Win detection is done by scanning the board for four in a row in every direction: across rows, down columns, and along both diagonals. The anti-diagonal case was tested separately from the main diagonal, since that's a spot where these kinds of bugs like to hide. A draw is simply detected when there are no legal moves left and nobody has won.

Because every agent talks to the engine through the same small set of functions, plugging in a new agent just means writing a class that asks the engine for legal moves and picks one. None of the agents need to know how the board is stored internally.

3. AI Agents

The random agent is the simplest of the three. It just asks the engine for the legal moves and picks one uniformly at random using a seeded random number generator, so the same seed always produces the same choice. This is the baseline that the other two agents are compared against.

The rule-based agent follows four priorities, checked in order. First, it checks if any legal move wins the game right away, and plays it if so. Second, if there's no immediate win, it checks whether the opponent has a move that would win for them next turn, and blocks it. Third, if neither of those apply, it narrows down to the legal moves that are closest to the center column, since central columns tend to open up more winning lines. Fourth, among those central moves, it looks at how long a line of its own discs each move would create in any direction, and picks the move that builds the longest line. Whenever two or more moves tie at any of these steps, the agent picks between them using a seeded random choice, so results stay reproducible.

The minimax agent is the most involved of the three. It searches the game tree using minimax with alpha-beta pruning, and for all the experiments reported here it was run at a fixed depth of

4. At a terminal state, a win is scored as a large fixed constant (1,000,000) plus the remaining search depth, and a loss as the negative of that same value, so that faster wins and slower losses are both preferred over equally-final but more distant outcomes; a draw scores zero. When the search hits the depth limit without a terminal state, it falls back to a heuristic that slides a length-four window across every row, column, and diagonal on the board, scoring each window based on how many of the agent's own discs versus the opponent's discs are in it. Windows with three of the agent's discs and an open space are weighted heavily (+100), and two in a row more modestly (+10), while the opponent's equivalent threats are penalized slightly more heavily than the agent's own are rewarded (−120 and −12 respectively), so that blocking a threat is weighted a bit above building one; there's also a small bonus for controlling the center column. Just like the other two agents, ties are broken with a seeded random pick, and moves are searched center-first to help alpha-beta pruning cut off more branches early.

4. Experimental Results

Each pairing was played over 30 games, with the two agents alternating who moves first (15 games each way). Every agent was constructed with its own seed for each individual game so the results are exactly reproducible. All numbers below come directly from a fresh end-to-end run of evaluate.py, verified twice for consistency, with the full 28-test unit test suite passing beforehand.

Test machine: Intel Core i7-13620H (2 logical CPUs allocated), 3.8 GB RAM, Ubuntu 22.04.5 LTS (Linux 6.8.0, x86_64), Python 3.10.12. All timing numbers were collected on this single machine in one sitting.

Random vs Rule-Based

Random seeded 1000–1029, Rule-Based seeded 2000–2029.

Agent	Win rate	Draw rate	Avg decision time (ms)
Random	1/30 (3.33%)	0/30 (0.00%)	0.0012
Rule-Based	29/30 (96.67%)	0/30 (0.00%)	0.2555

Rule-Based vs Minimax

Rule-Based seeded 3000–3029, Minimax seeded 4000–4029, depth 4.

Agent	Win rate	Draw rate	Avg decision time (ms)
Rule-Based	3/30 (10.00%)	1/30 (3.33%)	0.2719
Minimax	26/30 (86.67%)	1/30 (3.33%)	33.4204

Minimax vs Random

Minimax seeded 5000–5029, Random seeded 6000–6029.

Agent	Win rate	Draw rate	Avg decision time (ms)
Minimax	30/30 (100.00%)	0/30 (0.00%)	42.5260
Random	0/30 (0.00%)	0/30 (0.00%)	0.0021

5. Discussion

The results follow the expected strength ordering: Random weakest, Rule-Based in the middle, Minimax strongest. Rule-Based beat Random in 29 of 30 games, since immediate win/block detection plus center preference is already a large step up from picking moves at random; the single Random win shows the baseline isn't completely toothless. Minimax then dominated Rule-Based, winning 26 of 30 and drawing 1, with 3 losses. Against Random, Minimax swept all 30 games.

Looking directly at the per-game logs turned up a more specific finding than a generic restatement of the “Connect Four is solved” sanity check. In the Rule-Based vs Minimax pairing, every single game where Minimax moved first ended in exactly 9 moves, all 15 of them, and Minimax won every one. That's a clean, consistent signature rather than a coincidence, consistent with the fact that the first player in Connect Four can force a win with correct play, and depth-4 minimax combined with the heuristic is evidently strong enough to execute a fixed forcing sequence against this particular opponent. It's worth noting this specific 9-move pattern was tied to facing Rule-Based; in the Minimax vs Random pairing, Minimax's first-mover wins varied in length (7 to 17 moves), since Random's unpredictable defense doesn't allow the same forced line every time. Separately, all of Rule-Based's wins and its one draw against Minimax happened only in the games where Rule-Based moved first — it never won or drew from the second-mover position — reinforcing how much first-move advantage matters at this depth.

The decision time numbers tell a related story. Random and Rule-Based both decide in a fraction of a millisecond since neither looks ahead at all. Minimax takes noticeably longer, averaging around 33 to 43 ms per move, since it's expanding a full depth-4 game tree with pruning on every turn. Minimax was slower on average against Random (42.53 ms) than against Rule-Based (33.42 ms), which likely comes down to Random's scattered, unpredictable play opening up more branches for Minimax to consider, while Rule-Based's more purposeful play steers the game into positions where alpha-beta pruning can cut things off sooner.

For future improvement, a few directions stand out. Increasing the search depth further (say to 6) would likely make Minimax even stronger, though at a real cost to speed. Adding a transposition table would help avoid re-searching board positions that come up more than once through different move orders. And longer term, moving away from fixed-depth minimax toward something like Monte Carlo Tree Search could improve both playing strength and time efficiency, especially in the busier middle part of the game where the number of possible moves is largest.

6. Team Member Contributions

- Safdar Md Abdus Shukur: Requirement 1 (Game Engine) and Requirement 2 — Agent 1 (Random Agent).

- Morad Mohammad: Requirement 2 — Agent 2 (Rule-Based Agent).
- Noah Samarita: Requirement 2 — Agent 3 (Minimax Agent).
- Farhan Chowdhury: Requirement 3 (Experimental Evaluation) and the Report (Requirement 4).
- Manahil Bashir: Requirement 3 (Experimental Evaluation), done together with Farhan on his system, and Requirement 5 (Demonstration Video).